



Fullstack Web Development

Session 1 | Week 3/Day 2 | 120 min

Git

Instructor: Ms. Liev Nova



090667393



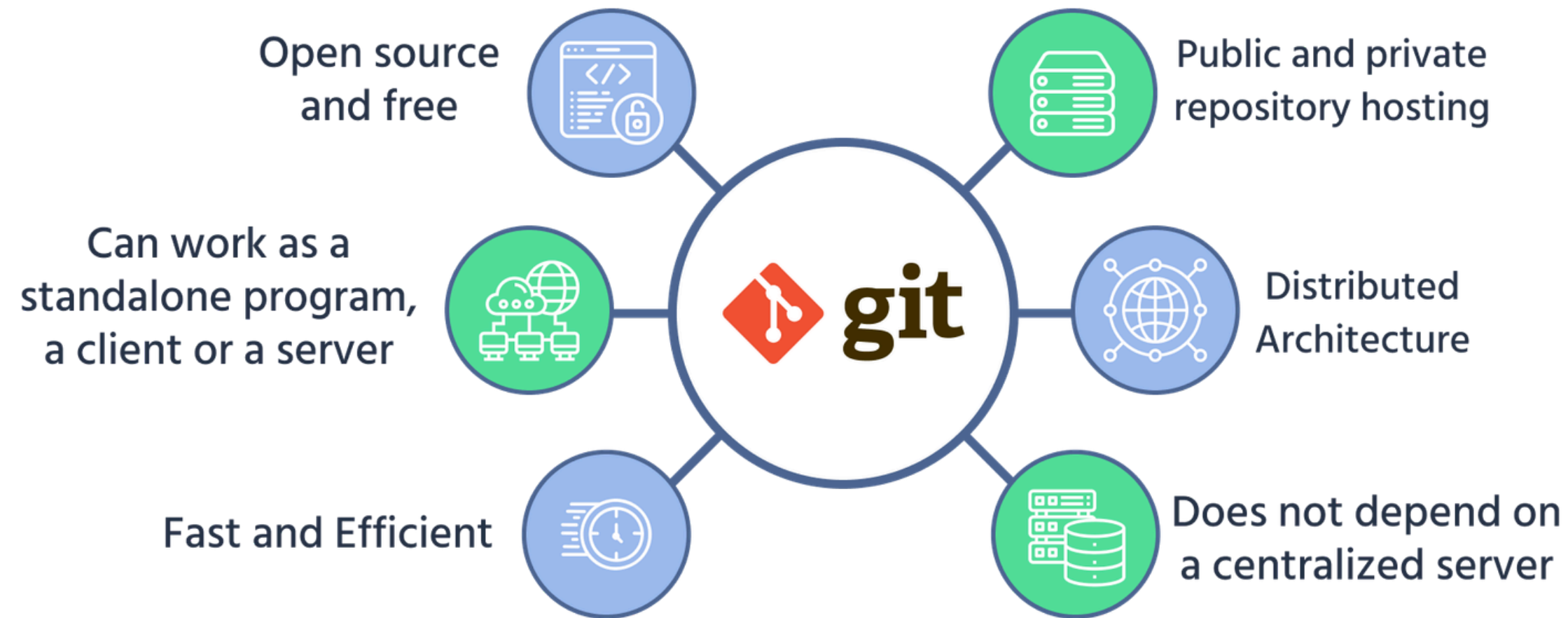
Git

What is **Git** and
How Does it Work?



What is Git?

- Git is a tool for **tracking changes** in files.
- Keeps **history of your project**.
- Enables **collaboration** with multiple developers.

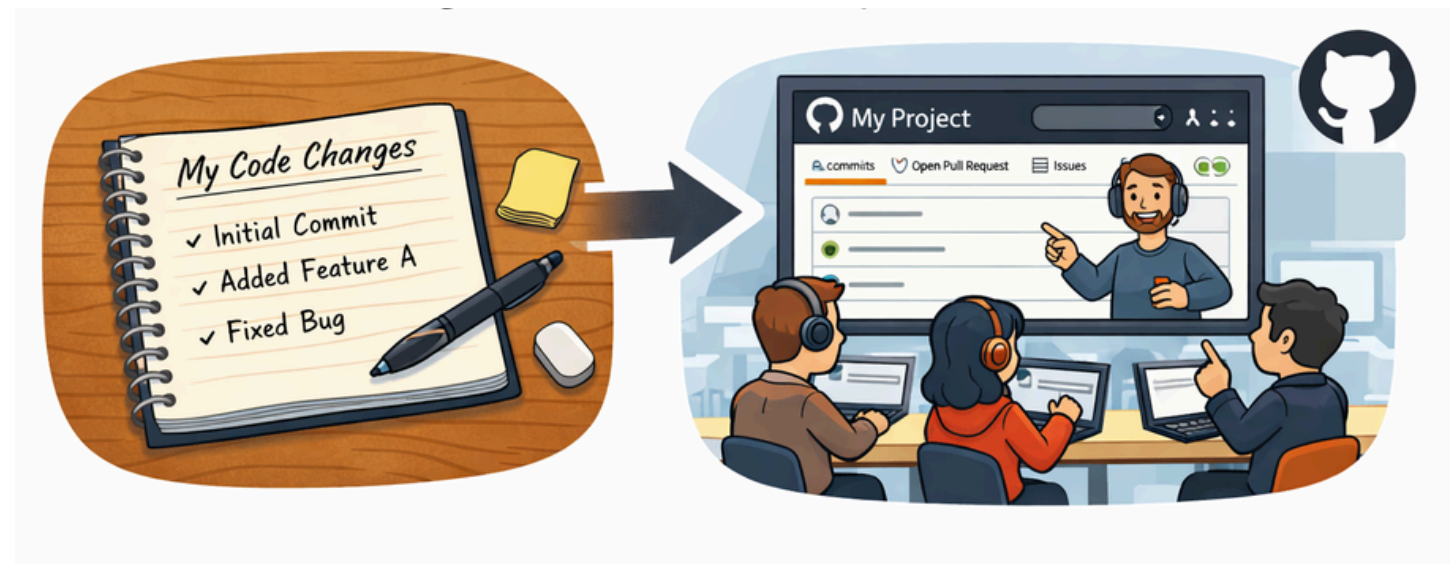


Git vs GitHub

Git: Version control software installed locally.



GitHub: Cloud service to store Git repositories online.



*You can use Git locally **without GitHub**, but GitHub allows **sharing & collaboration**.*

Installing Git

STEP01: Go to <https://git-scm.com/>

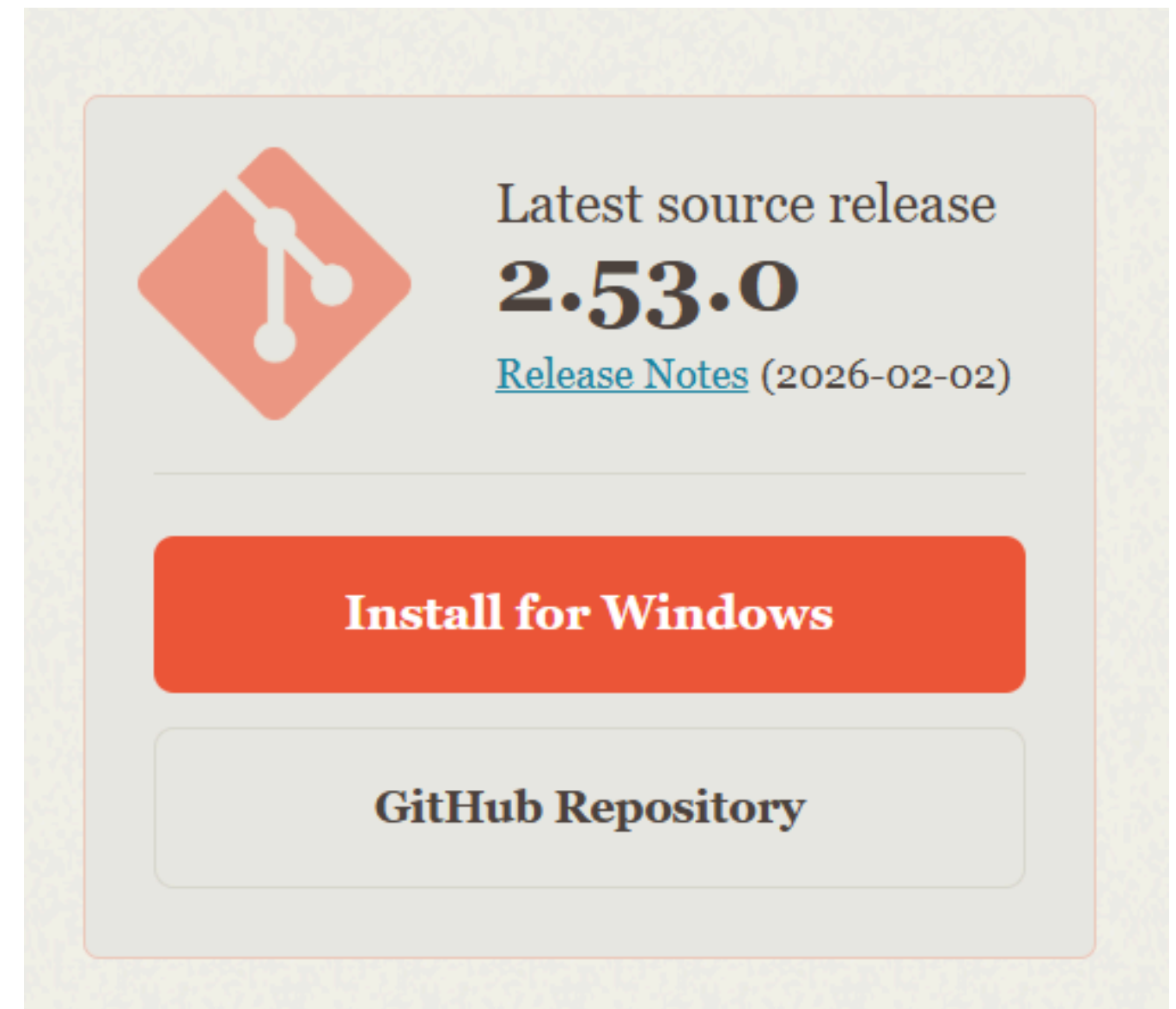
STEP02: Download Git for your OS (Windows, Mac, Linux)

STEP03: Verify installation: `git --version`

```
Command Prompt
Microsoft Windows [Version 10.0.26200.7840]
(c) Microsoft Corporation. All rights reserved.

C:\Users\lievnova>git --version
git version 2.51.0.windows.1

C:\Users\lievnova>
```



Setting Up Git

Configure your identity:

STEP01 : Open Git Bash

```
lievnova@DESKTOP-VEQKCA6 MINGW64 ~  
$ git config --global user.name "Liev Nova"
```

git config --global user.name "Liev Nova"

```
lievnova@DESKTOP-VEQKCA6 MINGW64 ~  
$ git config --global user.email "lievnova04@mail.com"
```

git config --global user.email "lievnova04@mail.com"

Check config:

```
lievnova@DESKTOP-VEQKCA6 MINGW64 ~  
$ git config --list
```

```
user.name=Liev Nova  
user.email=lievnova04@mail.com
```

Basic Git Workflow

STEP01: Initialize Repository: `git init`

- Creates a new Git repository in your project folder.
- Starts tracking changes in this folder.

STEP02: Check Status: `git status`

- Shows the current state of your files.
- Tells you which files are untracked, staged, or modified.
- Helps avoid mistakes before committing.

Basic Git Workflow

STEP03: Add Files to Stage: `git add filename.txt` or `git add .`

- Moves your changes into the staging area.
- Only staged files will be included in the next commit.

Note:

- `git add filename.txt` : add specific file
- `git add .` : add all

Basic Git Workflow

STEP04: Commit Changes: `git commit -m "Add initial files"` or `git commit Add_Iniial_Files`

- Saves a snapshot of staged changes in the Git history.
- Each commit should have a meaningful message explaining the changes.

STEP05: View Commit History: `git log`

- Shows all previous commits with messages, author, and timestamps.
- Helps you review what has been done or revert if needed.

Connect to GitHub

STEP 1 – Create a Repository on GitHub

- Go to [GitHub.com](https://github.com)
- Click New Repository
- Name your repository

1

General

Owner *

 lievnova ▾

/

Repository name *

Week01_Day02

✔ Week01_Day02 is available.

Great repository names are short and memorable. How about [miniature-umbrella?](#)

Description

0 / 350 characters

Connect to GitHub

STEP 1 – Create a Repository on GitHub

- Go to [GitHub.com](https://github.com)
- Click New Repository
- Name your repository

2

Configuration

Choose visibility *

Choose who can see and commit to this repository

 Private ▾

Add README

READMEs can be used as longer descriptions. [About READMEs](#)

Off ☐

Add .gitignore

.gitignore tells git which files not to track. [About ignoring files](#)

No .gitignore ▾

Add license

Licenses explain how others can use your code. [About licenses](#)

No license ▾

Connect to GitHub

SSTEP 1 – Create a Repository on GitHub

- Go to [GitHub.com](https://github.com)
- Click New Repository
- Name your repository

3

Jumpstart your project with Copilot (optional) 

Tell [Copilot coding agent](#) what you want to build in this repository. After creation, Copilot will open a pull request with generated files - such as a basic app, starter code, or other features you describe - then request your review when it's ready.

Prompt

Describe what you want Copilot to build

0 / 30000 characters

Create repository

Connect to GitHub

STEP 2 – Link Local Repo to GitHub :

`git remote add origin https://github.com/username/repository.git`

- Connects your local Git repository to a remote repository on GitHub.
- “**origin**” is the default name for the remote repository.

```
git init
git add README.md
git commit -m "first commit"
git branch -M main
git remote add origin https://github.com/lievnova/Week01_Day02.git
git push -u origin main
```


Connect to GitHub

STEP 3 – Push Changes: `git push -u origin main`

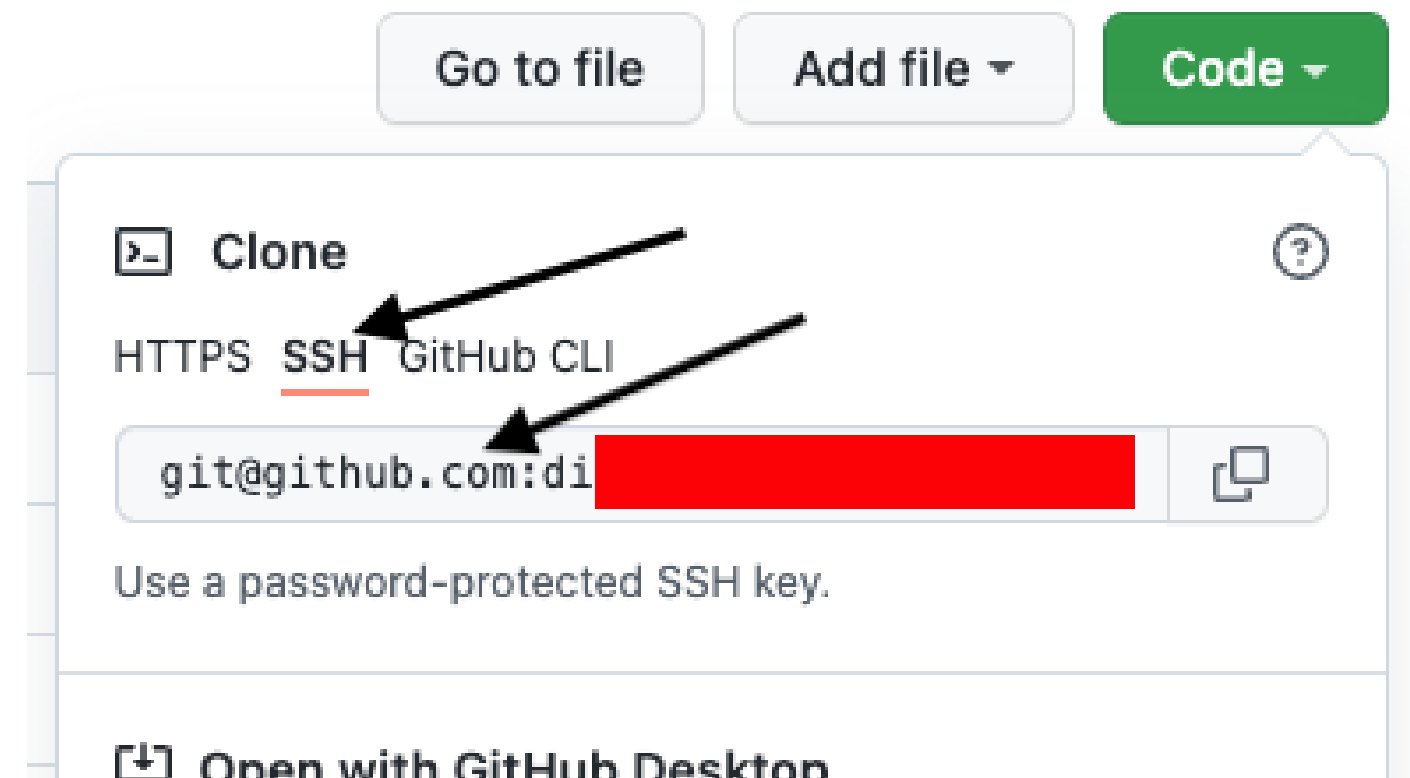
- Uploads your local commits to GitHub.
- The -u flag sets the upstream branch, so future git push works easily.
- Replace main with your branch name if different.

```
git init
git add README.md
git commit -m "first commit"
git branch -M main
git remote add origin https://github.com/lievnova/Week01_Day02.git
git push -u origin main
```

Cloning a Repository

Clone project : `git clone <repository-url>`

- Downloads a copy of an existing repository from GitHub to your local computer.
- Includes all files, branches, and commit history of the project.



Pulling Changes from GitHub

Pulling Changes from GitHub: `git pull origin main`

- Downloads the latest changes from GitHub to your local repository.
- Ensures it is **up-to-date** before making new changes.
- `main` is name of Branch
- You can switch to a different branch

Branching

Create a Branch : `git branch new-feature` or `git checkout -b new-feature`

- `git branch new-feature`
 - Creates a branch called new-feature.
 - Does not switch to it automatically. You remain on the current branch (e.g., main).
 - Think of it as preparing a new notebook page for a specific topic, but not writing on it yet
- `git checkout -b new-feature`
 - Combines both steps:
 - Creates the new-feature branch
 - Switches to it immediately
 - Saves time and avoids mistakes of forgetting to switch.

Merge

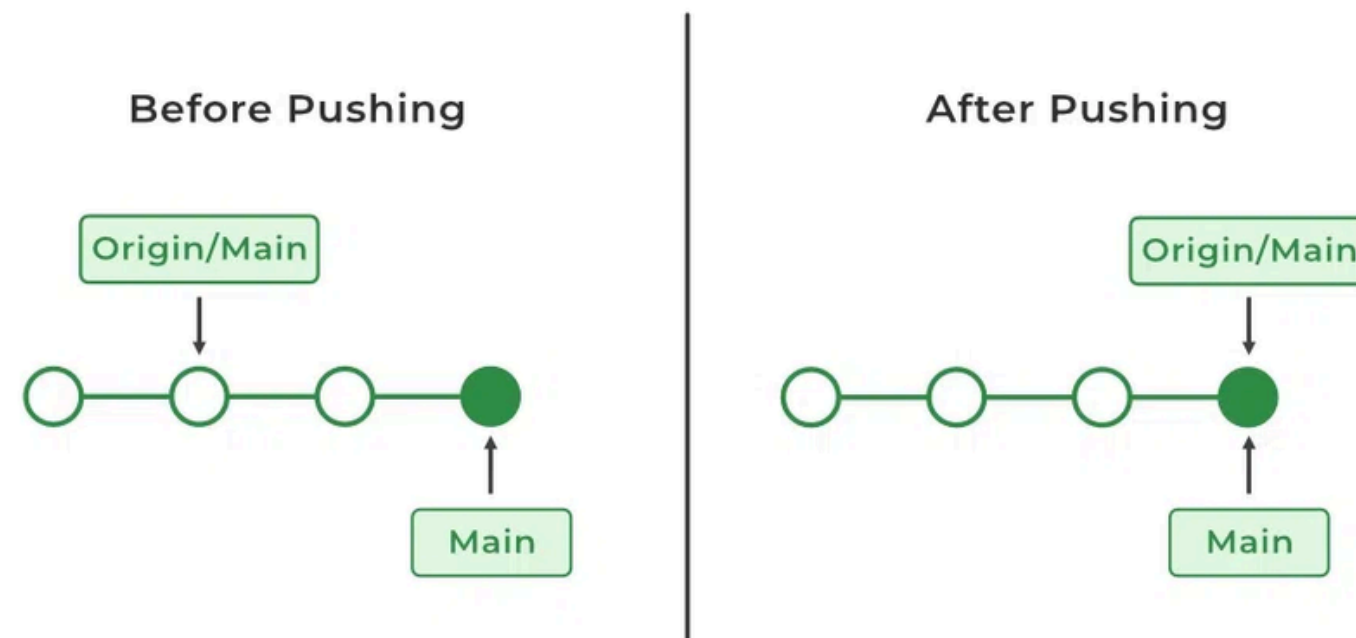
Merge code : `git merge <branch-name>`

- Switch to the branch you want to merge into (main).
- Merge the changes from new-feature into main.
- Git will try to automatically combine changes.
 - If changes conflict, Git will ask you to resolve conflicts manually.

Push to GitHub

Push code GitHub: `git push` or `git push origin main`

- Sends your local commits to the remote repository on GitHub.
- Makes your changes visible to others or accessible from another computer.



Resolving Conflicts

- A merge conflict happens when two branches modify the same line of a file or Git cannot automatically decide which change to keep.

```
<<<<<< HEAD
```

```
This is the code from the main branch
```

```
=====
```

```
This is the code from the feature branch
```

```
>>>>>> new-feature
```

- Steps to Resolve Conflict
 - Open the file with the conflict.
 - Decide which code to keep or combine both.
 - Remove the conflict markers (<<<<<<, =====, >>>>>>).
 - Save the file.

Ignoring Files

Sometimes you don't want certain files to be tracked by Git

Create file : `.gitignore`

```
# Common files to ignore
*.log
node_modules/
.DS_Store
.env
```

- `node_modules/` → Ignore dependency folder
- `.env` → Ignore environment variables file
- `*.log` → Ignore all log files
- `.DS_Store` → Ignore macOS system file



THANK YOU

For your attention !

